



SWIGGY

SQL Case Study

A Data Analyst's exploration of customer, restaurant & delivery insights using SQL

Presented by Samarth Pathak

Data Analyst Portfolio Project

Objective

This case study uses a Swiggy-style food delivery database — spanning customers, restaurants, orders, payments, and delivery partners — to answer real business questions with SQL. Each query below was written, executed, and validated against sample data to surface actionable insight, from customer behaviour to restaurant performance and delivery efficiency.

DATABASE AT A GLANCE

- **customers**
Customer profiles & cities
- **restaurants**
Restaurant listings & ratings
- **orders**
Order transactions
- **payment**
Amounts paid per order
- **deliverypartners**
Delivery partner roster
- **orderdelivery**
Order-to-partner mapping

SQL Techniques Used

1

Filtering & Sorting

WHERE, ORDER BY, LIMIT to narrow and rank results

2

Aggregation

COUNT, SUM, AVG paired with GROUP BY

3

HAVING Clauses

Filtering aggregated groups (e.g. counts > 1)

4

Joins

INNER & LEFT JOIN / USING across related tables

5

Subqueries

NOT EXISTS and correlated subqueries

6

Self Joins

Comparing customers against each other via aliases

7

Date Functions

DATE_ADD, CURRENT_DATE, YEAR for time-based filters

8

DISTINCT

Removing duplicate rows in multi-table results

CUSTOMER & RESTAURANT FUNDAMENTALS

Q1

Display all customers who live in 'Delhi'.

```
SELECT customer_id, name
FROM customers
WHERE city = "Delhi";
```

	customer_id	name
▶	2	Rohini Verma
	5	Manish Kumar
	18	Sonali Mishra

Simple WHERE filter returns every customer record whose city column matches 'Delhi'.

Q2

Find the average rating of all restaurants in 'Mumbai'.

```
SELECT city, AVG(rating)
FROM restaurants
GROUP BY city
HAVING city = "Mumbai";
```

	city	avg(rating)
▶	Mumbai	4.300000

GROUP BY city aggregates ratings; HAVING isolates the Mumbai group specifically.

CUSTOMER & RESTAURANT FUNDAMENTALS

Q3

List all customers who have placed at least one order.

```
SELECT DISTINCT o.customer_id, c.name
FROM orders o
JOIN customers c USING (customer_id);
```

	customer_id	name
▶	1	Amit Sharma
	2	Rohini Verma
	3	Rajesh Gupta
	4	Sneha Mehta
	5	Manish Kumar
	6	Priya Singh
	7	Vikas Reddy

An inner JOIN naturally excludes customers with zero orders; DISTINCT removes duplicates.

Q4

Display the total number of orders placed by each customer.

```
SELECT c.customer_id, c.name, COUNT(o.order_id)
FROM customers c
LEFT JOIN orders o USING (customer_id)
GROUP BY c.customer_id, c.name;
```

	customer_id	name	COUNT(o.order_id)
▶	1	Amit Sharma	2
	2	Rohini Verma	3
	3	Rajesh Gupta	3
	4	Sneha Mehta	2
	5	Manish Kumar	4
	6	Priya Singh	3
	7	Vikas Reddy	3

LEFT JOIN keeps every customer (even with 0 orders) while COUNT tallies their orders.

REVENUE & RATINGS ANALYSIS

Q5

Find the total revenue generated by each restaurant.

```
SELECT r.restaurant_id, r.name, SUM(amount_paid)
FROM restaurants r
JOIN orders USING (restaurant_id)
JOIN payment p USING (order_id)
GROUP BY r.restaurant_id, r.name
ORDER BY r.restaurant_id;
```

	restaurant_id	name	SUM(amount_paid)
▶	1	Spice of India	1250.00
	2	Tandoori Flames	1200.00
	3	Biryani House	2700.00
	4	Curry Pot	1750.00
	5	Taste of Punjab	600.00
	6	Royal Biryani	0.00
	7	Coastal Delight	1650.00

Chaining restaurants → orders → payment links each paid amount back to its restaurant.

Q6

Find the top 5 restaurants with the highest average rating.

```
SELECT name, rating
FROM restaurants
ORDER BY rating DESC
LIMIT 5;
```

	name	rating
▶	Biryani House	4.80
	Paradise Biryani	4.80
	Lucknowi Nawabi	4.70
	Royal Biryani	4.70
	Flavours of Bengal	4.60

Sorting ratings in descending order and capping at 5 rows surfaces the top performers.

CUSTOMER BEHAVIOUR PATTERNS

Q7

Display all customers who have never placed an order.

```
SELECT customer_id, name
FROM customers
LEFT JOIN orders USING (customer_id)
WHERE order_date IS NULL;
```

	customer_id	name
▶	24	Sonal Kaur
	25	Vivek Malhotra
	26	Divya Iyer
	27	Rakesh Yadav
	28	Mona Sharma
	29	Sudha Pillai
	30	Gaurav Khanna

A LEFT JOIN with a NULL check finds customers with no matching row in orders at all.

Q8

Find the number of orders placed by each customer in 'Mumbai'.

```
SELECT c.customer_id, c.name, COUNT(o.order_id)
FROM customers c
JOIN orders o USING (customer_id)
WHERE c.city = 'Mumbai'
GROUP BY c.customer_id, c.name;
```

	customer_id	name	COUNT(o.order_id)
▶	1	Amit Sharma	2
	3	Rajesh Gupta	3
	19	Arjun Desai	2
	23	Ravi Singh	2

Filtering by city before grouping narrows the order count down to Mumbai customers only.

CUSTOMER BEHAVIOUR PATTERNS

Q9

Display all orders placed in the last 30 days.

```
SELECT order_id, order_date
FROM orders
WHERE order_date >= DATE_ADD(
    (SELECT MAX(order_date) FROM orders),
    INTERVAL -30 DAY
);
```

	order_id	order_date
▶	1	2024-08-01 00:00:00
	2	2024-08-02 00:00:00
	3	2024-08-04 00:00:00
	4	2024-08-01 00:00:00
	5	2024-08-03 00:00:00
	6	2024-08-06 00:00:00
	7	2024-08-03 00:00:00

A subquery anchors 'last 30 days' to the dataset's most recent order date, not today.

Q11

Find the customers who have placed orders on exactly three different days.

```
SELECT c.customer_id, c.name, COUNT(DISTINCT o.order_date)
FROM customers c
JOIN orders o USING (customer_id)
GROUP BY c.customer_id, c.name
HAVING COUNT(DISTINCT o.order_date) = 3;
```

	customer_id	name	COUNT(DISTINCT o.order_date)
▶	2	Rohini Verma	3
	6	Priya Singh	3
	8	Anjali Patel	3
	14	Nidhi Saxena	3
	15	Ashok Kumar	3
	18	Sonali Mishra	3

COUNT(DISTINCT order_date) measures unique ordering days, then HAVING pins it to exactly 3.

DELIVERY PARTNER PERFORMANCE

Q10

List all delivery partners who have completed more than 1 delivery.

```
SELECT dp.partner_id, dp.name, COUNT(od.order_id)
FROM deliverypartners dp
JOIN orderdelivery od USING (partner_id)
GROUP BY dp.partner_id, dp.name
HAVING COUNT(od.order_id) > 1;
```

	partner_id	name	COUNT(od.order_id)
▶	4	Suresh Reddy	6
	5	Anita Desai	4
	2	Ravi Kumar	5
	6	Rajesh Gupta	4
	3	Priya Patel	3
	9	Sneha Iyer	2
	1	Amit Sharma	2

HAVING filters grouped delivery counts, keeping only partners above a single delivery.

Q12

Find the delivery partner who has worked with the most different customers.

```
SELECT dp.partner_id, dp.name,
       COUNT(DISTINCT o.customer_id) total_customers
FROM deliverypartners dp
JOIN orderdelivery USING (partner_id)
JOIN orders o USING (order_id)
GROUP BY dp.partner_id, dp.name
ORDER BY total_customers DESC
LIMIT 1;
```

	partner_id	name	total_customers
▶	4	Suresh Reddy	6

Distinct customer counts per partner are ranked and trimmed to the single top partner.

Q13 Identify customer pairs in the same city who ordered from the same restaurant, on different dates.

```
SELECT c1.name customer1, c2.name customer2,
       c1.city city1, c2.city city2, restaurants.name
FROM customers c1
JOIN orders o1 ON c1.customer_id = o1.customer_id
JOIN orders o2 ON o2.restaurant_id = o1.restaurant_id
JOIN customers c2 ON c1.city = c2.city
      AND c1.name <> c2.name
      AND o2.customer_id = c2.customer_id
JOIN restaurants ON o1.restaurant_id =
restaurants.restaurant_id
WHERE o1.order_date <> o2.order_date;
```

A self-join on customers (via matching city + shared restaurant) surfaces overlapping diners.

QUERY RESULT

customer1	customer2	city1	city2	name
Manish Kumar	Sonali Mishra	Delhi	Delhi	Biryani House
Sonali Mishra	Manish Kumar	Delhi	Delhi	Biryani House
Sonali Mishra	Manish Kumar	Delhi	Delhi	Biryani House
Arjun Desai	Ravi Singh	Mumbai	Mumbai	Veggie Delight
Manish Kumar	Sonali Mishra	Delhi	Delhi	Biryani House
Ravi Singh	Arjun Desai	Mumbai	Mumbai	Veggie Delight

Key Takeaways

1

Loyal city clusters

Delhi and Mumbai customers show the highest concentration of repeat activity, making them prime targets for retention campaigns.

2

Revenue is uneven across restaurants

Total revenue per restaurant varies widely (₹0 to ₹2,700+), highlighting partners who may need onboarding or promotional support.

3

A small group drives frequent orders

Customers ordering on 3+ distinct days and partners handling 1+ deliveries stand out as the platform's most engaged users.

4

One delivery partner leads on reach

Suresh Reddy served the most distinct customers (6) and completed the most deliveries (6) — a standout performer worth studying.



Thank You

Swiggy SQL Case Study • Samarth Pathak

Data Analyst Portfolio Project